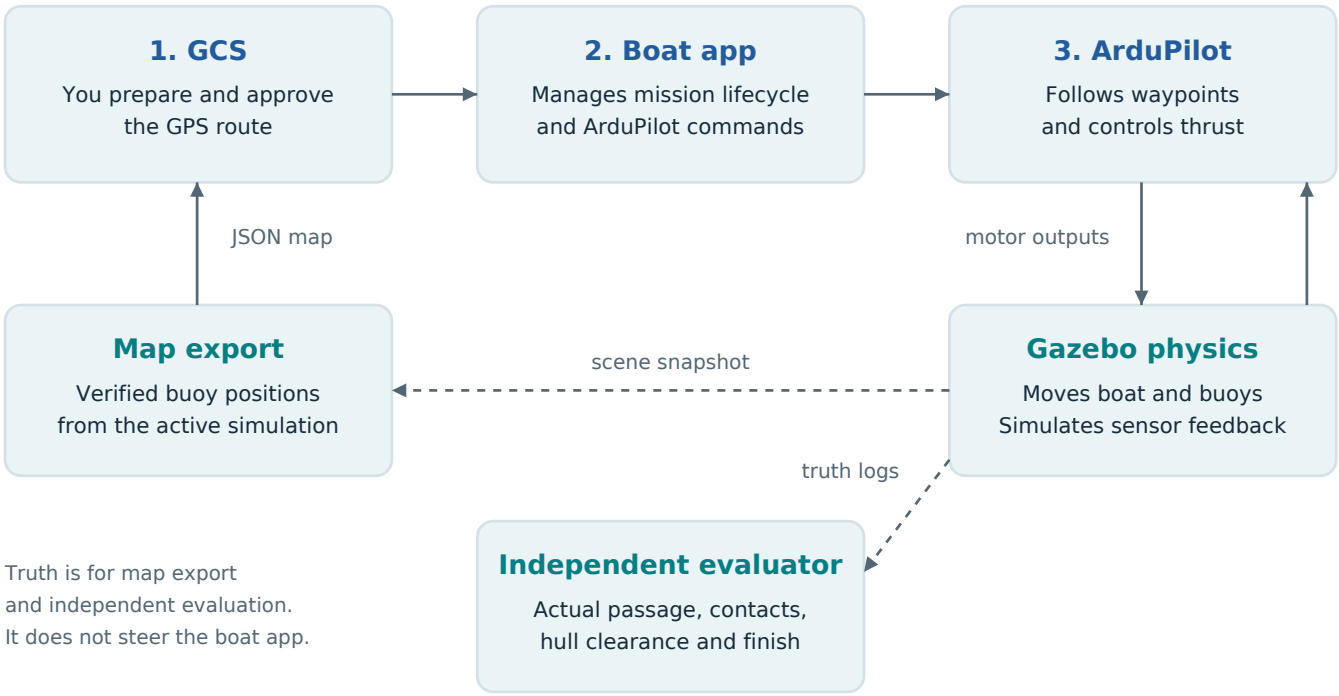
The system at a glance

A visual guide to data flow, ownership and every GCS action. Based on the repository code and documentation reviewed on 28 October 2026.

You choose the route. The GCS checks it. The boat app manages execution. ArduPilot drives.



What Milestone 1 demonstrates

Import the simulated buoy map, manually create a GPS route, validate and approve it, upload it through the boat app, and start a camera-free slalom. The course has an entrance gate, four turn buoys and an exit gate, followed by a finish area.

What is already recorded

The milestone document records three consecutive browser-started simulator passes at a **2.0 m/s commanded target**. The lowest independently measured whole-hull clearance was **1.26091 m**. Actual speed varies during braking and turns. These are historical results, not a new validation run.

Physical-boat performance, radio operation and vehicle-to-GCS mission networking remain unverified. Drone mapping and automatic route generation are deferred.

Who owns what

The GCS, boat app and simulator are separate application runtimes. Shared code is a library, not another running service.

Owner	Responsibility
Operator	Places waypoints; selects passage order, sides, speed, clearance and finish; explicitly approves, uploads and starts.
GCS browser React + Leaflet	Shows the map and live boat marker, edits the draft, sends actions and displays their results.
GCS backend FastAPI	Persists the workspace, checks the route, freezes approval and forwards commands to the boat HTTP API.
shared/	Common GPS/local-coordinate geometry and conservative planned-route validation.
Boat app ROS 2 + pymavlink	One command worker owns MAVLink. Manages upload/readback, start, progress, finish, stop and disarm. HTTP and ROS use this same worker.
ArduPilot	Stores and follows the waypoint mission, controls steering/thrust and reports vehicle state. Simulated now; Pixhawk on hardware.
Simulation tools	Export a verified scene map and independently assess the actual physics trajectory, contacts and hull clearance.
Drone	Future survey and geolocation work. It is a scaffold and has no active role in Milestone 1.

Three different data packages

Package	Contents and destination
Object map	Buoy IDs, categories, GPS centres, collision sizes, capture time and scene/alignment identity. Imported into the GCS.
Editable draft	Waypoints, planned start/heading, speed, clearance, finish, profile selection and passage order/sides. Stored by the GCS.
Mission envelope	Approval/revision/profile IDs plus GPS waypoints, speed, finish tolerances, timeout and starting-distance limit. Sent to the boat app.

The boat receives a path, not a buoy map. Left/right course rules stay in the GCS; ArduPilot follows the resulting waypoints.

Prepare the route

Editing prepares a draft. These controls do not change a mission that is already uploaded to ArduPilot.

Control	What it changes
Import object map	Reads exported JSON, saves the map and resets the draft/settings. Reimporting the same map also resets the route.
Map click / waypoint drag	Click appends a waypoint. Drag moves an existing waypoint.
Latitude / longitude	Edits a waypoint precisely in GPS degrees.
Up arrow / delete	Moves a waypoint earlier in the route / removes it.
Use boat as start	Copies the boat app's fresh reported position and heading into the draft. Does not move the boat.
Use last waypoint as finish	Sets the finish centre to the final waypoint. Validation requires these to match.
Save draft	Persists the editable workspace on the GCS backend. Does not approve, upload or arm.
OSM background	Toggles optional OpenStreetMap imagery. Editing remains available if the tiles cannot load.

Mission settings

Setting	Meaning
Speed / hull clearance	Requested mission speed / minimum hull-to-buoy gap. Speed must match the measured profile; clearance must be at least 0.25 m.
Finish radius / start heading	Allowed distance from the final waypoint, up to 1 m / initial orientation used to check the first leg.
Navigation profile	Measured evidence for speeds, hull envelope, tracking allowance, leg lengths, turns, firmware and controller settings. Selection does not automatically tune the vehicle.
Task / passage roles	Entrance-gate-only or full-slalom checks. An ordered JSON list identifies gate buoys and turn sides; it does not generate the route.

Left/right: relative to travel along the fixed course's +X direction, rather than relative to the screen or current boat heading. The current validator supports that course orientation.

Every draft edit revokes approval and clears the displayed validation result. Validate and Approve save pending valid edits first.

Validate, then approve

These actions happen in the GCS. Neither action uploads the mission or arms the boat.

Validate: does the planned route pass?

The backend calls shared/clearance.py. It checks map alignment, calibration evidence, supported speed, first-leg heading, leg lengths, turn angles, buoy clearance, passage order/sides and the finish region. Full slalom requires gate, four turns, then gate.

The route is checked from the planned start through every waypoint. The validator reports failures and the browser draws clearance margins. It does not move waypoints, generate a replacement route or reduce your requested clearance.

Why the safety margin is larger than the hull

Contribution	What it covers
Buoy radius	Distance from buoy centre to its collision surface.
Enclosing boat radius	A conservative hull envelope that can be used along planned line segments.
Measured tracking allowance	Observed positioning/path-following separation for the selected calibrated profile.
Extra 0.10 m reserve	An additional retained margin beyond the measured allowance.
Selected hull clearance	The minimum gap requested by the operator, at least 0.25 m.

Required boat-centre distance from buoy centre = buoy radius + boat radius + measured allowance + 0.10 m reserve + selected clearance.

This is a static planned-path test against empirical limits. Independent evaluation later checks the actual whole-hull motion, including turns.

Approve: freeze this exact version

Approve reruns validation. On success it saves an immutable record of the map, draft, measured profile, validation result and mission envelope. A revision hash identifies the approved inputs, and the approval ID becomes the mission ID.

Upload and Start verify that approval still matches the saved inputs and the current profile file. Editing a route, even back to its old values, revokes approval. Changing a profile file also invalidates the frozen revision.

No measured profile installed? Draft editing and saving still work; approval requires measured calibration evidence. No production profile is shipped in the repository.

Upload, start, stop, disarm

All mission commands travel browser -> GCS backend -> boat HTTP API at localhost:8091 -> the boat's command worker -> ArduPilot.

Upload: store and verify the approved mission

The GCS confirms approval and revalidates the path from the boat's **actual current position and heading**. The boat verifies the selected profile against live firmware, configured geometry and controller parameters; it requests and verifies the profile's position message rate.

Upload requires fresh vehicle data, a disarmed boat and a mode outside AUTO. The boat sends the waypoint items, waits for full ArduPilot acceptance, downloads the stored mission and compares it with the supplied route. Successful upload leaves the mission ready for a separate Start.

Start: set up execution and request arming

The GCS checks the actual start again. The boat requires a confirmed upload, fresh telemetry/GPS and acceptable starting distance, then rechecks the profile, stored mission and mission-end behavior.

Order	Boat action
1	Select HOLD and set the first route item.
2	Select AUTO while the vehicle is still disarmed.
3	Send target speed and wait for command acceptance.
4	Request normal arming and confirm the armed state.
5	Report executing; ArduPilot follows the uploaded waypoints.

Start requests arming. It does not force-arm or bypass ArduPilot pre-arm checks.

Stop and Disarm are separate

Stop: takes priority in the command queue, cancels pending work and requests HOLD. It works without a valid draft or approval, but requires a fresh autopilot heartbeat to confirm the mode change. HOLD does not guarantee an immediate physical stop or station keeping.

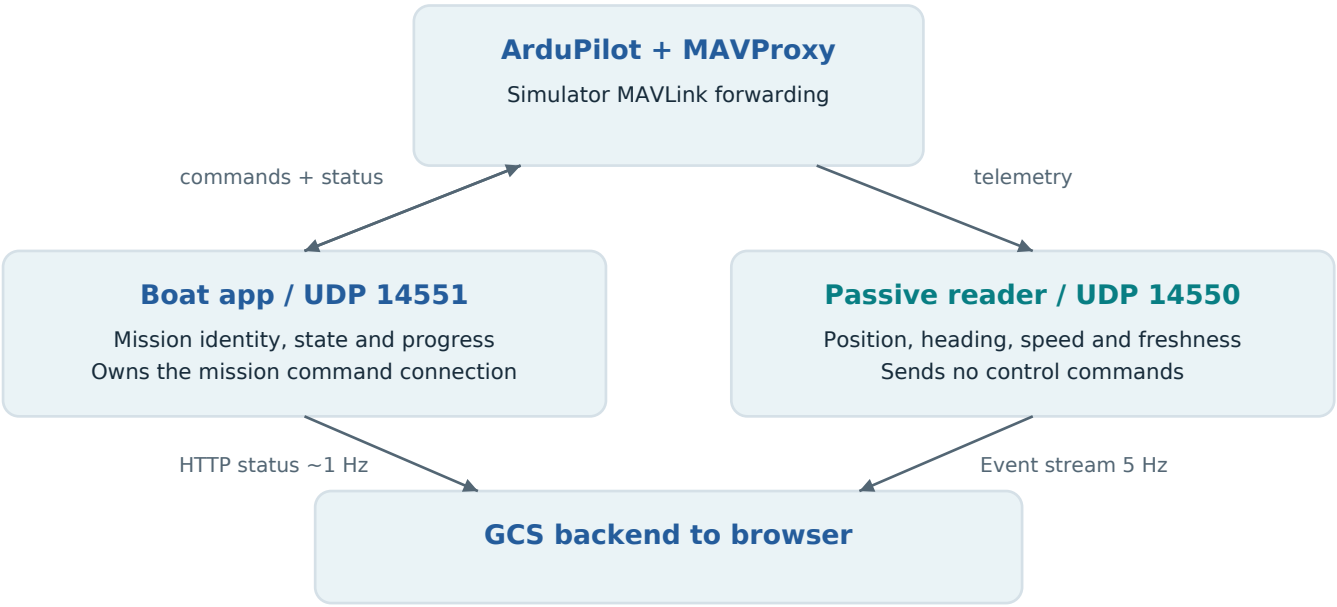
Disarm: requires fresh vehicle data, HOLD mode and reported speed at or below 0.2 m/s. The boat sends a normal disarm request and confirms it.

Request received is not vehicle acceptance. Follow the operation ID: pending -> running -> succeeded or failed. Start succeeded means execution began, not that the course is complete.

After an ambiguous timeout, inspect operation and vehicle status before retrying. Repeated requests for an identical mission ID are idempotent; deliberate new attempts use a new mission ID.

Watch the boat and finish

Two independent information paths feed the screen. A moving map marker alone does not prove that the approved mission is executing.



Mission status reports the mission ID, state, waypoint progress, mode, arming and operation outcomes. **Passive map telemetry** supplies the marker's position, heading and speed. The reader sends no commands, heartbeats or message-rate requests.

The map can work while the boat mission API is unavailable. Backend position freshness expires after 3 s without heartbeat or position; the browser also watches stream silence. A stale marker greys out at its last known position. There is no prediction or route snapping.

What happens at the last waypoint?

ArduPilot reports the final waypoint reached -> the boat app selects HOLD -> it checks position inside the finish radius and speed at or below **0.2 m/s continuously for five seconds** -> state becomes **complete**. Disarm remains an explicit operator action.

What happens if something disconnects?

Event	Behavior
Browser or GCS backend loss	The uploaded mission continues. Reconnecting resumes observation; it never automatically starts a mission.
Manual takeover / mode change	The boat marks execution interrupted and does not reclaim control. Autonomous execution needs an explicit new upload/start.
Boat telemetry failure	The boat marks failure and attempts HOLD only with fresh heartbeat and observed AUTO ownership. Total telemetry loss cannot establish a confirmed stop.
Boat process shutdown	Attempts bounded cleanup. Abrupt power/process loss cannot guarantee a stop; independent takeover remains essential.

What counts as success

Planned-route validation, boat completion and independent simulator acceptance answer different questions.

Result	What it establishes
Route validation passed	The planned path satisfies configured geometry, course rules and measured profile limits.
Boat mission complete	The last waypoint was reached and reported finish position/speed settled in HOLD.
Independent evaluator passed	Recorded physics confirms passage order/sides/stern clearance, no contacts, minimum whole-hull clearance and finish settling.

The evaluator uses per-step Gazebo poses and contacts with a conservative full-attitude collision footprint. The GCS does not launch it automatically when mission status says complete. Raw evidence is retained under ignored simulation/logs/.

Recorded acceptance and its limits

The selected simulator acceptance requires a 2.0 m/s target, at least 0.25 m whole-hull clearance, GPS alignment error at most 0.10 m, finish position error at most 1 m, speed at most 0.2 m/s for five seconds, a 300 s timeout and three consecutive passes with frozen inputs.

The recorded final series passed, including GCS backend loss/reconnect during the third run. The minimum clearance was 1.26091 m; actual mean speed was about 0.732 m/s and peak about 1.917 m/s on this braking/pivoting route. It does not establish continuous 2.0 m/s travel or real-world boat/GPS performance.

An operator view of the full sequence

Stage	What you should confirm
Prepare	Map imported; route and passage roles entered; speed/profile selected; start and finish set.
Authorize	Validate passes, then Approve freezes the intended route and settings.
Transfer	Upload operation succeeds after ArduPilot acceptance and mission readback.
Execute	Start operation succeeds and the boat reports executing. Watch mission state as well as the map marker.
Finish	Boat reports complete after HOLD and settling; Disarm is explicitly confirmed.
Evaluate	Independent simulator evidence establishes actual whole-hull and course acceptance.

A perfect simulator map does not model real survey error, buoy drift or physical boat dynamics. Later experiments must measure those effects before simulator margins can be treated as hardware limits.

Read the code in order

Start with the screen, follow its backend endpoint, then trace the boat worker into the MAVLink executor. The file paths below are relative to the repository root.

File or directory	Follow it to understand...
gcs/frontend/src/main.jsx	Every visible control, draft edits, status polling and map rendering.
gcs/app/main.py	Button endpoints, actual-start revalidation and boat HTTP forwarding.
gcs/app/workspace.py	Saved drafts, immutable approvals and mission-envelope creation.
shared/clearance.py	Margins, leg/turn limits, gate order, passing sides and finish checks.
boat/boat_app/control.py	The single command queue, STOP priority and durable operation IDs.
boat/boat_app/executor.py	MAVLink upload/readback, start sequence, HOLD, disarm and completion.
gcs/app/telemetry.py	Passive MAVLink filtering, freshness, reconnection and snapshots.
simulation/scripts/	export_active.py / export_map.py and evaluate_run.py.

Design and operating references

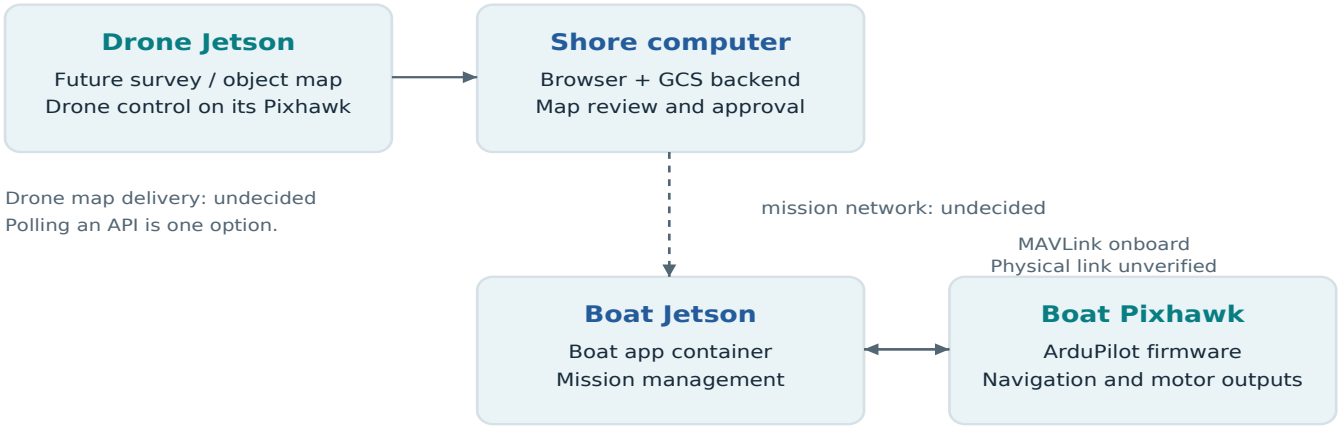
Source references: docs/design.md; docs/milestone1.md; docs/boat_execution.md; boat/README.md; gcs/README.md; and the implementation files above. This guide explains the 28 October 2026 review snapshot and is not a replacement for current operating instructions.

Testing the boat without the GCS

Independent test path: the boat also accepts a manually supplied JSON mission through its ROS services, without GCS or drone. That path does not perform GCS object-map clearance approval.

Software on each device

This is the intended hardware split. Today, the accepted simulator setup runs the GCS, boat app and simulated vehicle as separate containers on the development computer.



Device	Software and role
Shore computer	Browser: React/Leaflet interface. Local GCS container: FastAPI, workspace/approvals, route validation and passive MAVLink telemetry.
Drone Jetson Planned	Future survey processing, detection and GPS object-map production. Currently a scaffold; its API and push/poll delivery are undecided.
Drone Pixhawk Planned	Flight-controller software for drone control. Firmware, integration and deployment are unconfirmed; boat firmware/settings do not establish the drone setup.
Boat Jetson Target deployment	Boat app container: ROS 2 Jazzy, boat_mission node, HTTP API, command worker, executor, pymavlink and logs. Jetson deployment is unverified.
Boat Pixhawk	ArduPilot firmware outside Docker: stores/follows waypoints, processes navigation inputs and produces steering/thrust outputs.

How the devices are connected

Today: localhost:8080 serves the GCS; its backend calls 127.0.0.1:8091 on the boat app. Simulator MAVProxy forwards MAVLink to UDP 14551 for the boat app and UDP 14550 for passive GCS map telemetry. Host networking lets the separate containers use these host loopback addresses.

On hardware: shore-to-boat mission networking and drone-map delivery still need a selected transport. A boat Jetson-to-Pixhawk MAVLink connection is intended, but its physical wiring/device configuration is unverified. A radio telemetry link alone does not deliver the mission HTTP envelope to the Jetson.

localhost always means the current machine. The existing loopback-only boat API is not directly reachable from a separate shore computer; hardware mission delivery needs an explicit network design.

Inside the boat

The onboard computer manages the mission. The Pixhawk performs the continuous vehicle-control work. They have different jobs even though both are on the same boat.



On the Jetson: mission coordination

The container starts the Python ROS 2 node **boat_mission**, its HTTP server, local ROS command services, one worker and a pymavlink connection. ROS publishes status every 0.5 seconds. HTTP readers use status snapshots; only the worker reads MAVLink.

The worker queues commands and gives STOP priority. At upload/start it verifies the measured profile. The executor checks fresh vehicle data, upload/readback, modes, speed and arming, then monitors waypoints and finish settling.

ArduPilot messages update vehicle and mission status. Events and a durable operation journal record outcomes. Restarting the app does not restore start authorization or automatically resume a mission.

On the Pixhawk: navigation and actuation

ArduPilot stores the waypoints. Once AUTO, target speed and arming are confirmed, it uses navigation/sensor inputs to follow the route and command steering/thrust. In simulation, Gazebo moves the boat and returns sensor feedback. On hardware, the Pixhawk works with the physical sensors and motor-control hardware.

The Jetson does not continuously send steering setpoints for Milestone 1. It uploads and supervises a waypoint mission that ArduPilot executes.

What stays independent

No camera pipeline, buoy mapper, Nav2 planner or simulator-truth subscription is required aboard the boat for this milestone. Buoy locations, course-side rules and route approval remain in the GCS. The Jetson receives the approved route and execution limits.

Loss of the GCS link alone does not stop the uploaded mission. An independent operator takeover path through the Pixhawk is part of the design. Simulation verifies independent MAVLink mode takeover; physical RC/radio behavior remains unverified. Host drivers and Pixhawk firmware stay outside the application container.

Additional sources: docs/hardware.md; boat/container/compose.yaml; boat/boat_app/node.py and http_api.py. Physical deployment remains unverified.